

手摸手之列车余票如何保障缓存数据库一致性

查看本篇文档前，请先保证已阅读缓存一致性的理论文档。

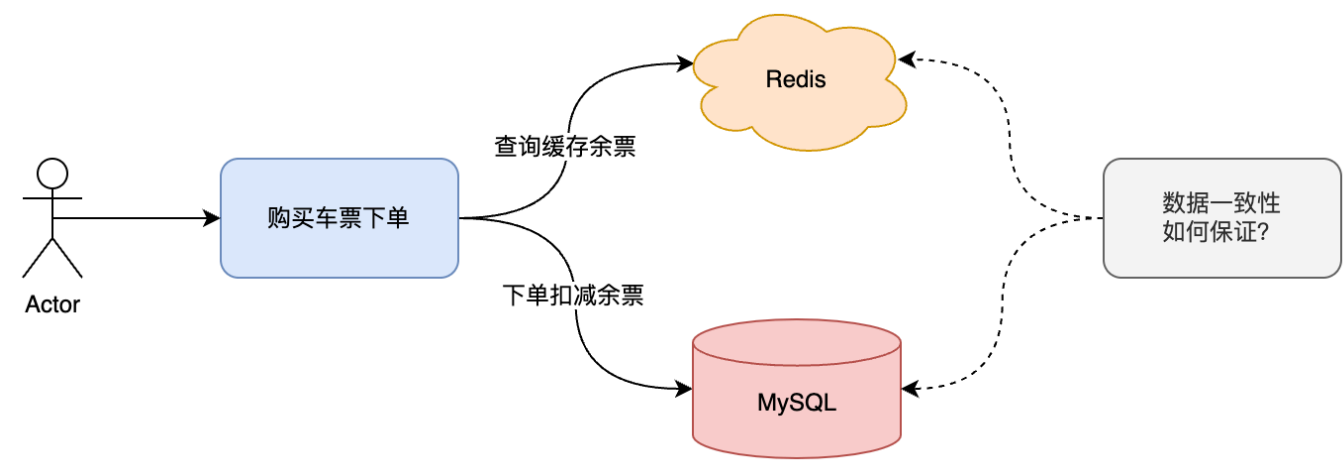
缓存与数据库一致性如何解决？

业务背景

为了满足用户对一趟列车不同站点不同座位类型的余量查询需求，我们采取了一种优化方案。我们将这些余量信息存储在缓存中，以使用户可以快速查询。

然而，在用户创建订单并完成支付时，我们需要同时从数据库和缓存中扣减相应的列车站点余票。这种设计不仅提高了查询效率，也保证了数据的一致性，确保订单操作的准确性。

在这个业务场景中的缓存与数据库一致性如何保证？



项目实战

通过数据库缓存一致性文章我们了解到，适合咱们这个车票余量缓存的方案是通过 Binlog 异步更新列车缓存，详细逻辑查看上文。

通过 Binlog 异步更新列车缓存，意味着需要先更新数据库，这样也意味着数据库会承受更多的压力？这里咱们先按下不表，因为这是在购票全流程中非常重要的逻辑，会单独写一篇文章给大家说明。

下文实战章节中我们以 12306 G35 趟复兴号高铁为大家举例。

北京 → 杭州 （ 08月20日 周天 ） 共计4个车次

车次	出发站	到达站	出发时间	到达时间	历时	商务座 特等座	一等座	二等座 二等包座
+ G35	北京南	杭州东	09:56	15:14	05:18			
+ G39	北京南	杭州东	09:56	15:14	05:18			
+ D717	北京南	杭州东	09:56	15:14	05:18			
+ Z284	北京南	杭州东	09:56	15:14	05:18			

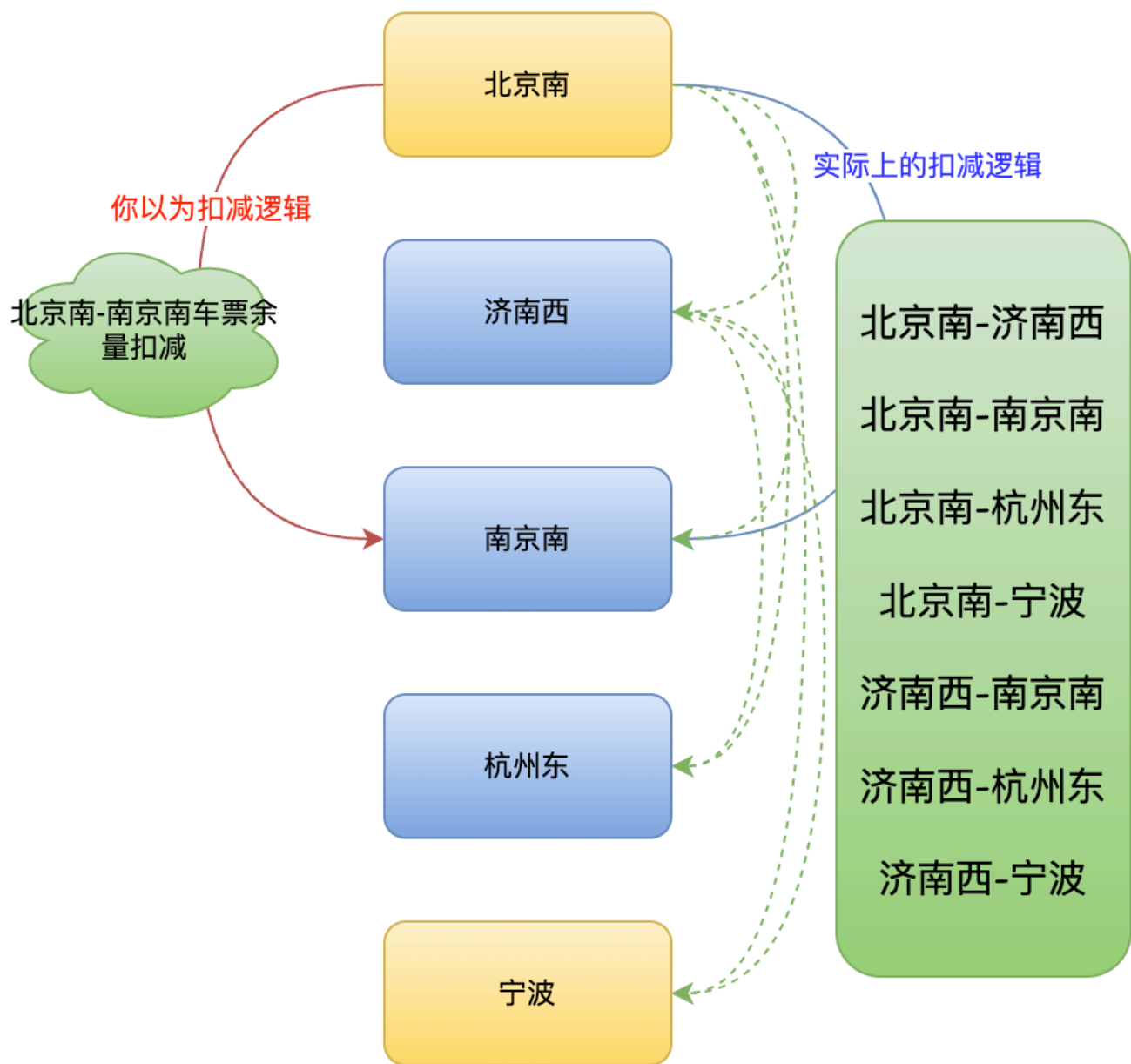
站序	站名	到站时间	出发时间	停留时间
01	北京南	09:56	09:56	
02	济南西	11:19	11:21	2
03	南京南	13:20	13:22	2
04	杭州东	14:26	14:28	2
05	宁波	15:14	15:14	

如果查询结果中没有符合您需求的车次，您还可以使用中转换乘功能，查询起讫点不连续的车次。显示的卧铺票价均为上铺票价，供您参考。具体票价以您确认支付时实际购买的铺别票价为准。如

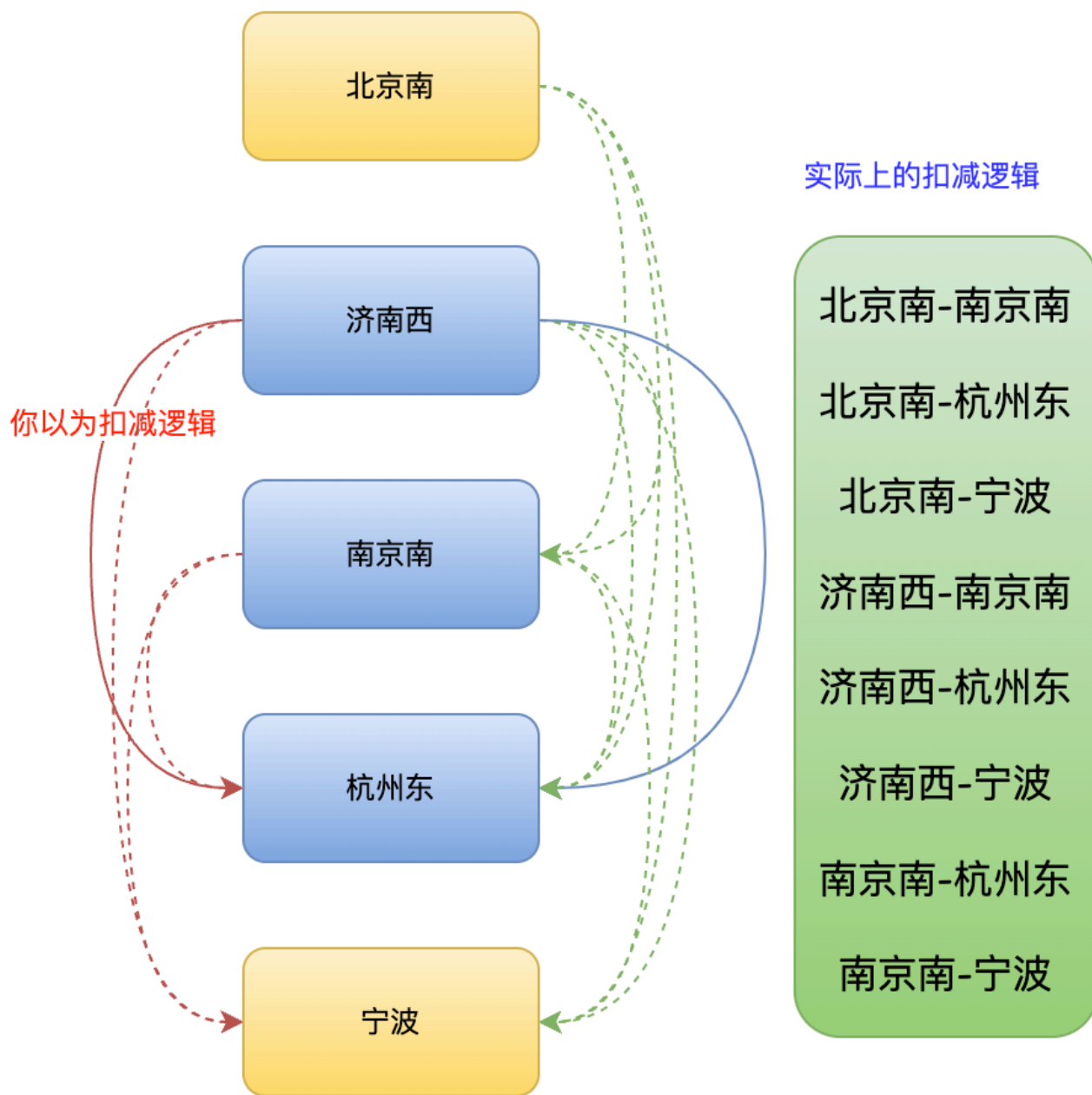
如何扣减列车库存余量

刚接触 12306 的大部分同学可能会误以为，如果购买了从北京南到南京南的车票，只会扣减北京南到南京南站点车票的库存。但实际情况并非如此。购买了北京南到南京南的列车票时，会扣减沿途站点的库存。

假设购买商务座，列车站点间的初始库存是一致的。下面是购买北京南到南京南车票的流程图示：



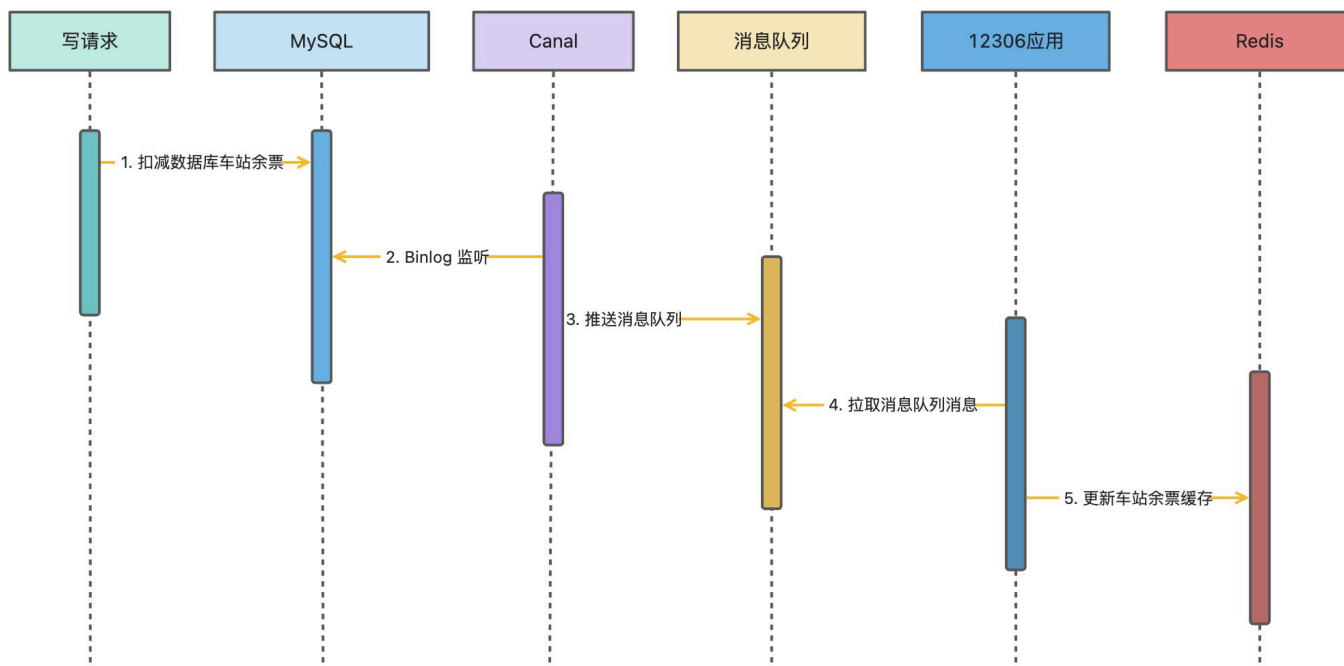
再来一个进阶版本的，如果购买济南西到杭州东的车票，流程图如下所示。大家可能会比较疑惑，为什么要扣减北京南到南京南、北京南到宁波、北京南到杭州东以及北京南到宁波？这是因为中间车票被购买了，北京南作为始发站，只能购买北京南到济南西的车票。



如何保证缓存一致性

我们采用了一种确保缓存和数据库一致性的方案，使用 Canal 监听 Binlog 模式。该方案将数据库的数据变更通过 Canal 转发给消息队列的特定 Topic。客户端应用程序可以监听该消息队列的 Topic，以保持缓存与数据库的一致性。

时序图如下所示：



这套逻辑在应用层面上非常严谨，但存在一个问题，即复杂性较高。如果大家想进行测试，需要启动 Canal 监听并结合消息队列进行测试。

为了解决这个问题，我在代码中添加了两种缓存同步模型。其中一种是在业务代码中同步地操作缓存扣减，这种方案比较简单，可以帮助大家快速熟悉业务流程。但是，这种方案在中间或快结束时如果出现异常，可能会导致缓存数据出现问题。

方案具体体现在购票抽象方法 `AbstractTrainPurchaseTicketTemplate#executeResp` 中添加了条件判断，如果是非 binlog 模式，也就是 `ticket.availability.cache-update.type` 不等于 binlog，就按照同步更新缓存模式进行。

```
@Override
public List<TrainPurchaseTicketRespDTO> executeResp(SelectSeatDTO requestParam) {
    List<TrainPurchaseTicketRespDTO> actualResult = selectSeats(requestParam);
    // 扣减车厢余票缓存，扣减站点余票缓存
    if (CollUtil.isEmpty(actualResult) &&
        !StrUtil.equals(ticketAvailabilityCacheUpdateType, "binlog")) {
        String trainId = requestParam.getRequestParam().getTrainId();
        String departure = requestParam.getRequestParam().getDeparture();
        String arrival = requestParam.getRequestParam().getArrival();
        String keySuffix = StrUtil.join("_", trainId, departure, arrival);
        StringRedisTemplate stringRedisTemplate = (StringRedisTemplate)
distributedCache.getInstance();
        stringRedisTemplate.opsForHash().increment(TRAIN_STATION_REMAINING_TICKET +
```

```
keySuffix, String.valueOf(requestParam.getSeatType()), -actualResult.size());
    }
    return actualResult;
}
```

另一种缓存同步模型被称为 Binlog 模式，它需要在配置文件中添加相关配置并启动 Canal 中间件才能启用。

如何开启 Binlog 缓存一致性

如果你启动的聚合模式，配置加在 aggregation-service 服务配置文件，分布式模式加在 ticket-service 服务配置文件。

```
ticket:
  availability:
    cache-update:
      type: binlog
```

对于自建 MySQL，需要先开启 Binlog 写入功能，my.cnf 中配置需要包含如下信息。

```
[mysqld]
log-bin=mysql-bin # 开启 binlog
binlog-format=ROW # 选择 ROW 模式
server_id=1 # 配置 MySQL replaction 需要定义，不要和 canal 的 slaveId 重复
```

配置修改成功后，重启一下 MySQL，确保所有的配置生效。然后执行 MySQL 执行命令查看是否修改 Binlog 成功。

```
# 查看 binlog 日志是否开启
show variables like 'log_%';
```

如果查询出的选型 log_bin 数据为 ON，Binlog 即为开启状态。

	Variable_name	Value
1	log_bin	ON
2	log_bin_basename	/var/lib/mysql/mysql-bin
3	log_bin_index	/var/lib/mysql/mysql-bin.index
4	log_bin_trust_function_creators	OFF
5	log_bin_use_v1_row_events	OFF
6	log_builtin_as_identified_by_password	OFF
7	log_error	stderr
8	log_error_verbosity	3

最后，MySQL 执行 SQL 语句创建 canal 单独使用的账号，用来进行 Binlog 的同步和监听。

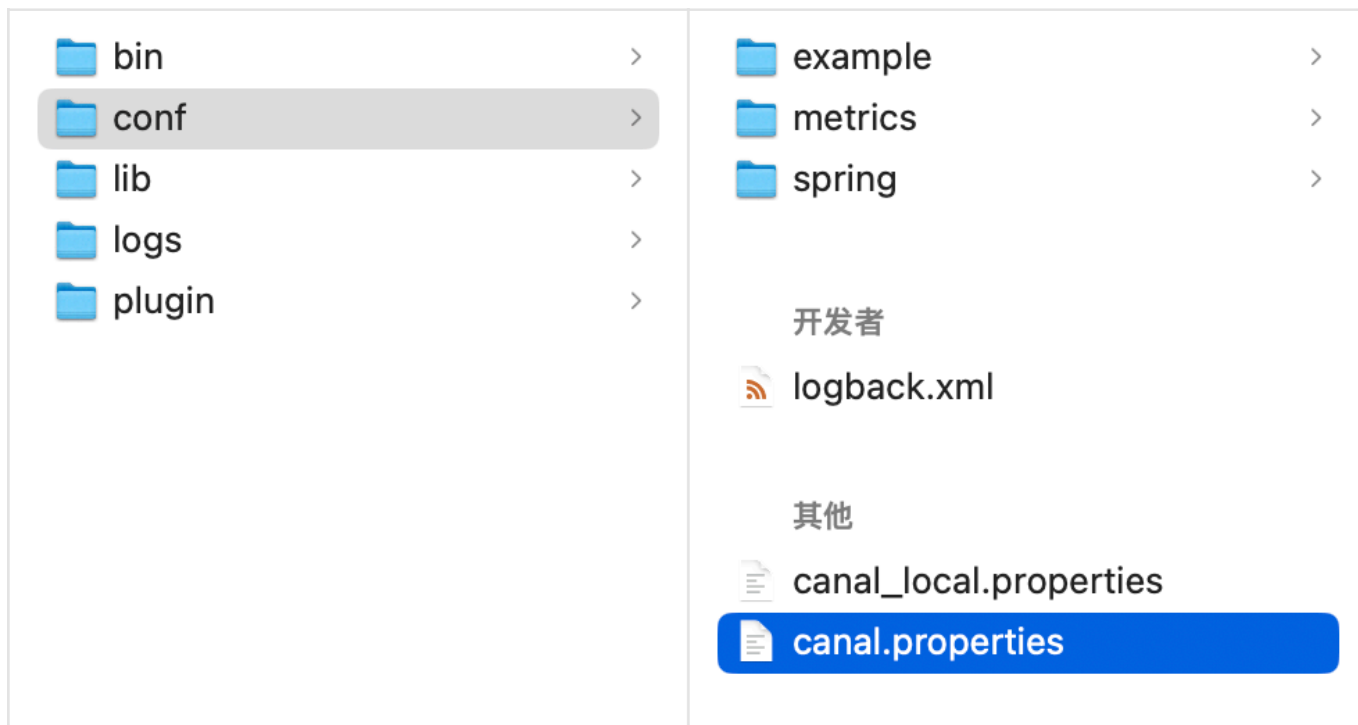
```
CREATE USER canal IDENTIFIED BY 'canal';
GRANT SELECT, REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'canal'@'%';
FLUSH PRIVILEGES;
```

接下来开始配置 Canal 中间件，下载 Canal 安装包，其中的一些配置都是我调试后的。

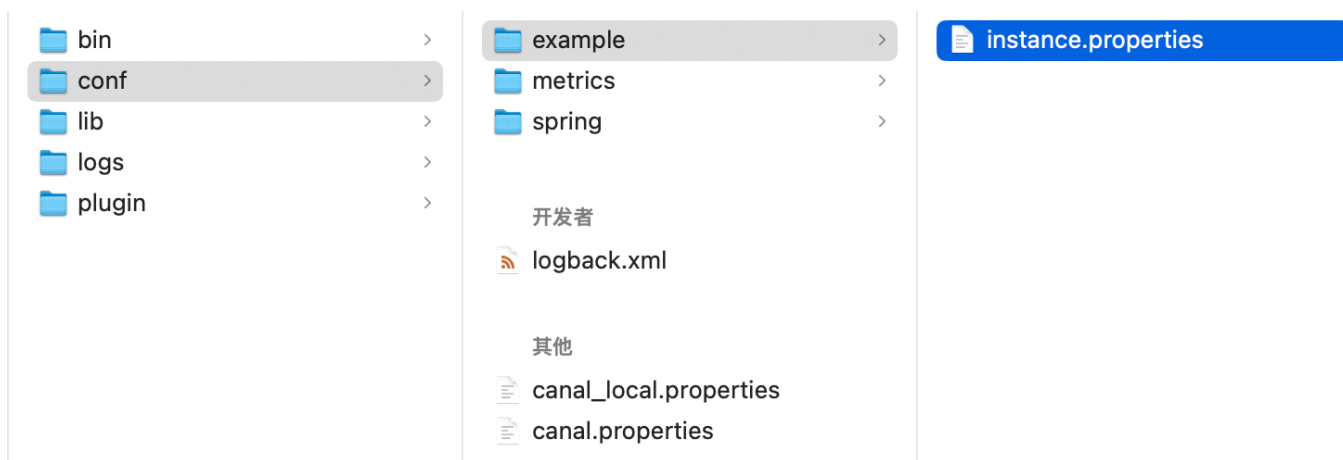
链接: <https://pan.baidu.com/s/1_8mm3LHXpxb6oFPbDpzfGQ?pwd=yhxt> 提取码: yhxt

注意，Canal 安装路径中不要有文件夹名称是中文。需要使用 JDK8 启动 Canal 服务，不要使用 JDK17。

如果你是自己本地启动的 RocketMQ，将 canal.properties 文件中的 rocketmq.namesrv.addr 配置项变更为自己本地的 RocketMQ 配置，默认为公有云中间件地址。



如果本地 MySQL 不是 `127.0.0.1:3306` 配置，需要将 `instance.properties` 配置文件中的 `canal.instance.master.address` 配置项替换为真实的 MySQL 地址和端口。



另外，如果你正在使用公有云 RocketMQ，`instance.properties` 配置文件中的 `canal.mq.topic` 也需要变更，配置中默认为 `index12306_canal_ticket-service_common-sync_topic-mading`，`-mading` 需要大家变更为自己项目在启动 VM 参数里的值。各自检查并进行替换。

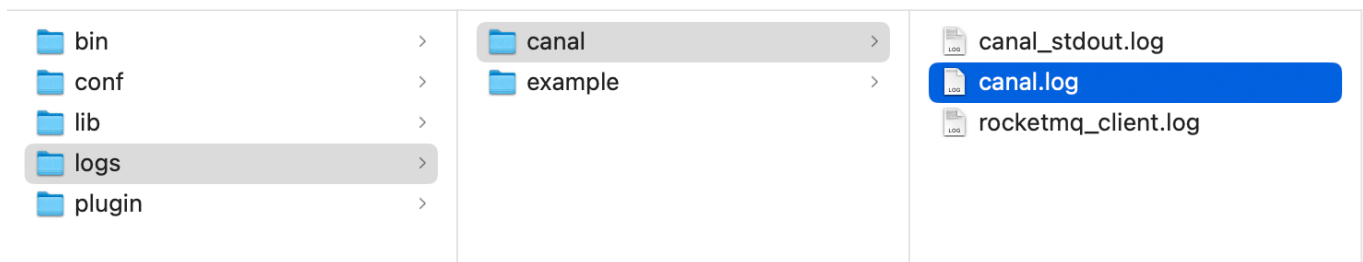

```
Build and run Modify options ⌵

java 17 SDK of 'index' ⌵ -cp index12306-aggregation-service

-Dspring.data.redis.password=Sm9sVXB0YjI030b5tz0trjpvZzRhtZmEbv0u0ImcD1wED0PfeaqI
-Dspring.data.redis.port=19389
-Dunique-name=-mading
-Dframework.cache.redis.prefix=mading:
-Dspring.data.redis.host=common-redis-dev.magestack.cn
-Drocketmq.name-server=common-rocketmq-dev.magestack.cn:9876
```

全部上述配置大家全都搞定后，开始启动 Canal 进行测试。如果是 Windows 系统，直接双击 Canal 文件夹下 bin 目录的 `startup.bat` 脚本即可，要停止 Canal 关闭脚本弹框。如果是 Linux 或者 Mac 系统，进入 bin 目录下执行 `sh startup.sh` 命令。如果是 Linux 或者 Mac 系统要停止执行 `sh stop.sh`。

启动完成后，打开 logs 文件夹目录下的 canal/canal.log 文件。



查看是否启动成功，如果输出打印如下 `the canal server is running now`，即为成功。

```
2023-08-20 15:27:18.106 [main] INFO com.alibaba.otter.canal.deployer.CanalLauncher - ## set default uncaught exception handler
2023-08-20 15:27:18.110 [main] INFO com.alibaba.otter.canal.deployer.CanalLauncher - ## load canal configurations
2023-08-20 15:27:18.153 [main] INFO c.a.o.c.c.rocketmq.producer.CanalRocketMQProducer - ##Start RocketMQ producer##
2023-08-20 15:27:18.198 [main] INFO com.alibaba.otter.canal.deployer.CanalStarter - ## start the canal server.
2023-08-20 15:27:18.214 [main] INFO com.alibaba.otter.canal.deployer.CanalController - ## start the canal server[192.168.31.96(192.168.31.96):11111]
2023-08-20 15:27:18.843 [main] INFO com.alibaba.otter.canal.deployer.CanalStarter - ## the canal server is running now .....
```

然后，我们变更数据库 `t\_seat` 表中任意一条记录的 `seat\_status` 值，比如从 0 变为 1，或者从 1 变为 0。

接下来查看 RocketMQ 控制台。

RocketMq-console-ng

页面配置查询如下，选择好主题和开始结束时间（默认会填充）后，点击搜索按钮。

RocketMQ控制台 运维 驾驶舱 集群 主题 消费者 生产者 消息 消息轨迹 更换语言

TOPIC MESSAGE KEY MESSAGE ID

Only Return 2000 Messages

选择自己在配置文件中配置的 Topic

主题: index12306_canal_ticket-service_common-... 开始: 2023-08-20 14:31 结束: 2023-08-20 16:31 搜索

注意开始时间和结束时间是否包含你变更数据的时间

Message ID	Tag	Key	StoreTime	Operation
7F00000107E7520A3426652E7F3A0000			2023-08-20 15:32:26	MESSAGE DETAIL

这条记录就是咱们刚才在数据库中变更的记录 binlog 产生的消息

打开查看详细消息体

消息体格式为 JSON 类型数据，示例数据如下：

```
{
  "data": [
    {
      "id": "1684913289981231104",
      "train_id": "1",
      "carriage_number": "01",
      "seat_number": "02C",
      "seat_type": "0",
      "start_station": "北京南",
      "end_station": "南京南",
      "price": "186400",
      "seat_status": "1",
      "create_time": "2023-07-28 21:06:47",
      "update_time": "2023-08-17 15:41:00",
      "del_flag": "0"
    }
  ],
  "database": "12306",
  "es": 1692516745000,
  "id": 2,
  "isDdl": false,
  "mysqlType": {
    "id": "bigint(20) unsigned",
    "train_id": "bigint(20)",
    "carriage_number": "varchar(64)",
    "seat_number": "varchar(64)",
    "seat_type": "int(3)",
    "start_station": "varchar(256)",
    "end_station": "varchar(256)",
```

```

        "price":"int(11)",
        "seat_status":"int(3)",
        "create_time":"datetime",
        "update_time":"datetime",
        "del_flag":"tinyint(1)"
    },
    "old":[
        {
            "seat_status":"0"
        }
    ],
    "pkNames":[
        "id"
    ],
    "sql":"",
    "sqlType":{"
        "id":-5,
        "train_id":-5,
        "carriage_number":12,
        "seat_number":12,
        "seat_type":4,
        "start_station":12,
        "end_station":12,
        "price":4,
        "seat_status":4,
        "create_time":93,
        "update_time":93,
        "del_flag":-6
    },
    "table":"t_seat",
    "ts":1692516746008,
    "type":"UPDATE"
}

```

对应配置字段信息的语意如下：

```

package org.opengooify.index12306.biz.ticketsevice.mq.event;

import lombok.Data;

import java.util.List;

```

```
import java.util.Map;

/**
 * Canal Binlog 监听触发时间
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Data
public class CanalBinlogEvent {

    /**
     * 变更数据
     */
    private List<Map<String, Object>> data;

    /**
     * 数据库名称
     */
    private String database;

    /**
     * es 是指 Mysql Binlog 里原始的时间戳，也就是数据原始变更的时间
     * Canal 的消费延迟 = ts - es
     */
    private Long es;

    /**
     * 递增 ID，从 1 开始
     */
    private Long id;

    /**
     * 当前变更是否是 DDL 语句
     */
    private Boolean isDdl;

    /**
     * 表结构字段类型
     */
    private Map<String, Object> mysqlType;
```

```
/**
 * UPDATE 模式下旧数据
 */
private List<Map<String, Object>> old;

/**
 * 主键名称
 */
private List<String> pkNames;

/**
 * SQL 语句
 */
private String sql;

/**
 * SQL 类型
 */
private Map<String, Object> sqlType;

/**
 * 表名
 */
private String table;

/**
 * ts 是指 Canal 收到这个 Binlog, 构造为自己协议对象的时间
 * 应用消费的延迟 = now - ts
 */
private Long ts;

/**
 * INSERT (新增)、UPDATE (更新)、DELETE (删除) 等等
 */
private String type;
}
```

监听 Canal 推送的消息 Topic

正如前文所述，Canal 会将 Binlog 的变更内容推送到指定的 RocketMQ Topic。因此，在 Spring Boot 应用中，我们只需要与 RocketMQ 进行对接即可。

```
package org.opengoofy.index12306.biz.ticketservice.mq.consumer;

import cn.hutool.core.collection.CollUtil;
import cn.hutool.core.util.StrUtil;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.rocketmq.spring.annotation.RocketMQMessageListener;
import org.apache.rocketmq.spring.core.RocketMQListener;
import org.opengoofy.index12306.biz.ticketservice.common.constant.TicketRocketMQConstant;
import org.opengoofy.index12306.biz.ticketservice.common.enums.CanalExecuteStrategyMarkEnum;
import org.opengoofy.index12306.biz.ticketservice.mq.event.CanalBinlogEvent;
import org.opengoofy.index12306.framework.starter.designpattern.strategy.AbstractStrategyChoose;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.util.Objects;

/**
 * 列车车票余量缓存更新消费端
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Slf4j
@Component
@RequiredArgsConstructor
@RocketMQMessageListener(
    topic = TicketRocketMQConstant.CANAL_COMMON_SYNC_TOPIC_KEY,
    consumerGroup = TicketRocketMQConstant.CANAL_COMMON_SYNC_CG_KEY
)
public class CanalCommonSyncBinlogConsumer implements RocketMQListener<CanalBinlogEvent>
{

    private final AbstractStrategyChoose abstractStrategyChoose;
```

```

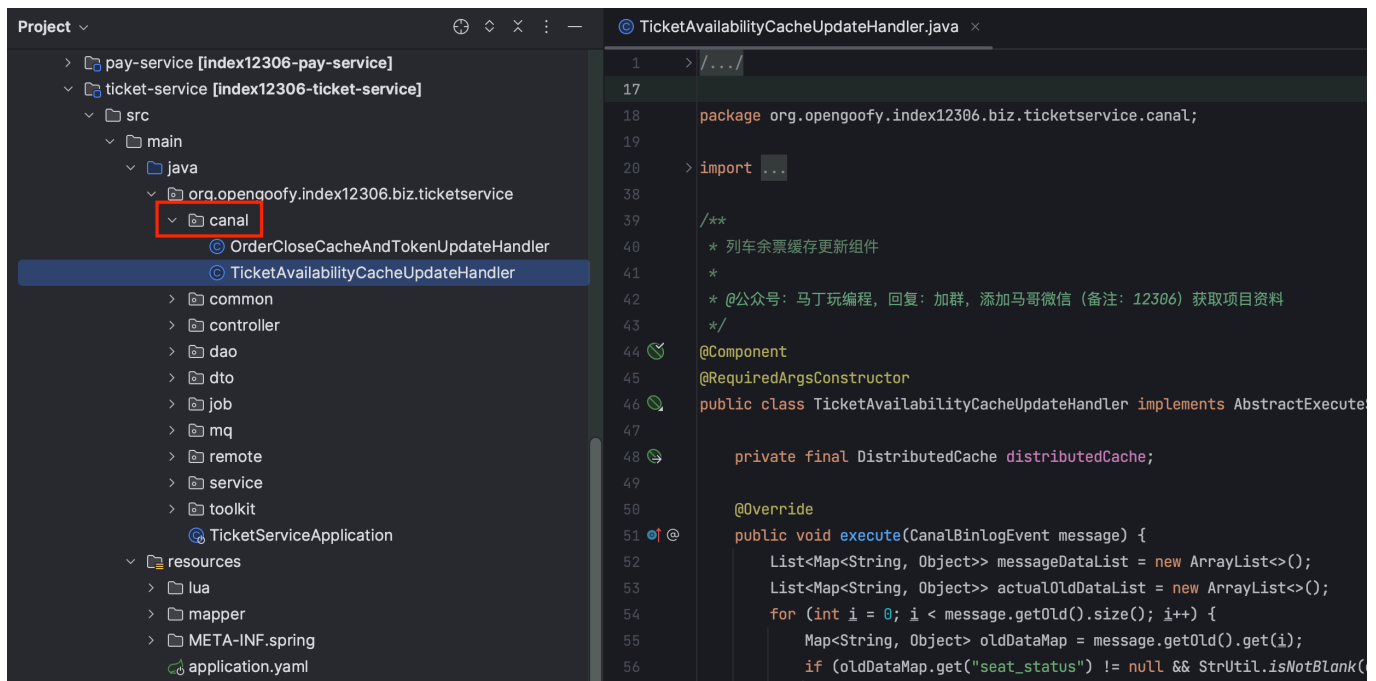
@Value("${ticket.availability.cache-update.type}")
private String ticketAvailabilityCacheUpdateType;

@Override
public void onMessage(CanalBinlogEvent message) {
    // 如果是 DDL 返回
    // 如果不是 UPDATE 类型数据变更返回
    // 如果没有开启 binlog 数据同步模型返回
    if (message.getIsDdl()
        || CollUtil.isEmpty(message.getOld())
        || !Objects.equals("UPDATE", message.getType())
        || !StrUtil.equals(ticketAvailabilityCacheUpdateType, "binlog")) {
        return;
    }
    // 通过策略模式进行不同 Binlog 变更类型的监听，比如说订单和座位两个表就分别有两个处
    理类
    abstractStrategyChoose.chooseAndExecute(
        message.getTable(),
        message,
        CanalExecuteStrategyMarkEnum.isPatternMatch(message.getTable())
    );
}
}

```

负责 Canal RocketMQ 消费者取名叫做 `CanalCommonSyncBinlogConsumer` 寓意着我们进行监听的所有表的数据修改 Binlog 都会通过这个消息队列进行监听。

那如果监听了多个表怎么办？分发模式是通过策略模式以及表名称找到对应的策略实现类。具体的策略实现类在 canal 目录下。



如果对策略模式不熟悉的同学，查看以下文档熟悉策略模式以及抽象后的策略模式使用。

摊牌了！策略模式在项目设计中用得最多

死磕设计模式之如何抽象策略模式

接下来重点解析 `TicketAvailabilityCacheUpdateHandler` 列车余票缓存更新策略实现类。

先来看第一步数据过滤逻辑。

为何进行数据过滤？我们明白，若监听了 `t_seat` 表的 Binlog 数据变化，即使对任意字段进行修改，都会引发变更流程，从而将数据传送至 RocketMQ 消息队列，随后客户端会消费此消息。但实际关注的是 `seat_status` 字段的改变，例如由 0（可售状态）转变为 1（锁定状态），或者从 1 变为 0。只有这种数据变化才需要更新相应的余票缓存，其他字段的变化并不需要触发后续逻辑。因此，在此阶段，若非预期的数据变动，可直接过滤。

```

List<Map<String, Object>> messageDataList = new ArrayList<>();
List<Map<String, Object>> actualOldDataList = new ArrayList<>();
for (int i = 0; i < message.getOld().size(); i++) {
    Map<String, Object> oldDataMap = message.getOld().get(i);
    if (oldDataMap.get("seat_status") != null &&
    StrUtil.isNotBlank(oldDataMap.get("seat_status").toString())) {
        Map<String, Object> currentDataMap = message.getData().get(i);

```



```

        if (StrUtil.equalsAny(currentDataMap.get("seat_status").toString(),
String.valueOf(SeatStatusEnum.AVAILABLE.getCode()),
String.valueOf(SeatStatusEnum.LOCKED.getCode())) {
            actualOldDataList.add(oldDataMap);
            messageDataList.add(currentDataMap);
        }
    }
}
if (CollUtil.isEmpty(messageDataList) || CollUtil.isEmpty(actualOldDataList)) {
    return;
}

```

第二步，更新列车站点余票缓存逻辑。

检测座位状态是否在 0（可售状态）和 1（锁定状态）之间进行变化。当从 0 到 1 变化时，表示生成了订单需要减少库存。若由 1 变为 0，则表示订单被取消或超时关闭，需要增加列车库存。

因此，判定自增或自减库存，我们仅需考虑旧数据的 seatStatus 是否为 0。若为 0，则表示新数据已变更为 1，需要自减列车站点库存；反之，自增库存。

由于 Binlog 可能一次触发多个数据变更，我们将这些操作整理至一个 Map，以进行统一操作。为方便逻辑查阅，我已在代码中对缓存进行增减。如追求最大性能，应将所有操作放入 LUA 脚本中，或通过管道命令执行，以最大化性能表现。

```

Map<String, Map<Integer, Integer>> cacheChangeKeyMap = new HashMap<>();
for (int i = 0; i < messageDataList.size(); i++) {
    Map<String, Object> each = messageDataList.get(i);
    Map<String, Object> actualOldData = actualOldDataList.get(i);
    String seatStatus = actualOldData.get("seat_status").toString();
    int increment = Objects.equals(seatStatus, "0") ? -1 : 1;
    String trainId = each.get("train_id").toString();
    String hashCacheKey = TRAIN_STATION_REMAINING_TICKET + trainId + "_" +
each.get("start_station") + "_" + each.get("end_station");
    Map<Integer, Integer> seatTypeMap = cacheChangeKeyMap.get(hashCacheKey);
    if (CollUtil.isEmpty(seatTypeMap)) {
        seatTypeMap = new HashMap<>();
    }
    Integer seatType = Integer.parseInt(each.get("seat_type").toString());
    Integer num = seatTypeMap.get(seatType);
    seatTypeMap.put(seatType, num == null ? increment : num + increment);
}

```

```
cacheChangeKeyMap.put(hashCacheKey, seatTypeMap);  
}  
StringRedisTemplate instance = (StringRedisTemplate) distributedCache.getInstance();  
cacheChangeKeyMap.forEach((cacheKey, cacheVal) -> cacheVal.forEach((seatType, num) ->  
instance.opsForHash().increment(cacheKey, String.valueOf(seatType), num)));
```

问题答疑Q\&A

- 为什么订阅 Canal 的消费者不用线程池?
- 为什么扣减数据库需要扣减沿途车站, 而缓存扣减不需要?
- 为什么创建订单扣减库存而不是支付后扣减库存?
-

详细内容请查看下述知识星球主题。

[知识星球 | 深度连接铁杆粉丝, 运营高品质社群, 知识变现的工具](#)

更新: 2024-09-09 19:41:11

原文: <<https://www.yuque.com/magestack/12306/glv5e0785b2d7oag>>